Tabelas de dispersão (*hashing*)

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Algoritmos e Estruturas de Dados II
Prof. Fabrício Sérgio de Paula

Tópicos

- Introdução
- Funcionamento
- Funções de dispersão
- Tratamento de colisões por encadeamento
- Tratamento de colisões por endereçamento aberto
- Exercícios

Introdução

- Tabela de dispersão: estrutura de dados para busca, inserção e remoção de chaves
  - Função de dispersão: associa a cada chave um valor numérico
    - Valor numérico é usado como índice em tabela (vetor) para acessar dados
  - Complexidade da busca, inserção e remoção pode ser O(1)
    - O(n) no pior caso

Funcionamento

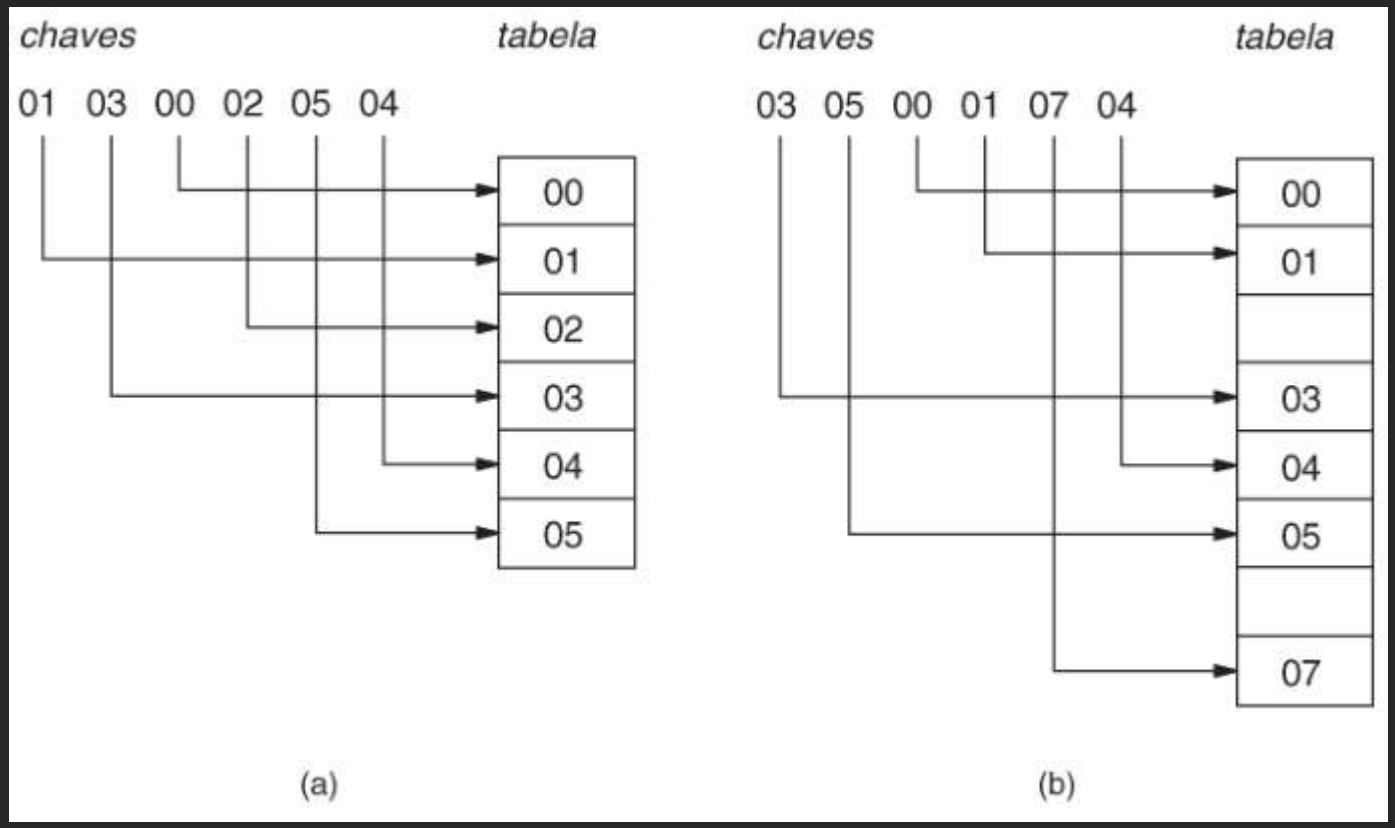
- Sejam n chaves a serem armazenadas em uma tabela sequencial T de dimensão m
  - Tabela tem m compartimentos: endereços de 0 até m-1
  - Cada compartimento pode armazenar r chaves distintas
    - Tabela em memória: cada compartimento armazena 1 chave
    - Tabela em disco: compartimento armazena diversos chave
      - Quantidade de chaves que cabem em um bloco do disco
    - Daqui em diante: cada compartimento armazena 1 chave

Funcionamento

- Caso bastante simples: n = m e valores das chaves de 0 a m-1
  - O valor de cada chave é usado diretamente como índice da tabela (chave x no compartimento x)
  - Técnica denominada acesso direto
    - Bastante empregada em programação comercial
    - Também pode ser utilizada quando n < m, com m – n pequeno
      - Chaves ainda estarão no intervalo de 0 a m-1: haverá m-n compartimentos vazios (espaços)

Funcionamento

- Endereçamento direto com n = m (a) e n < m (b)



Funcionamento

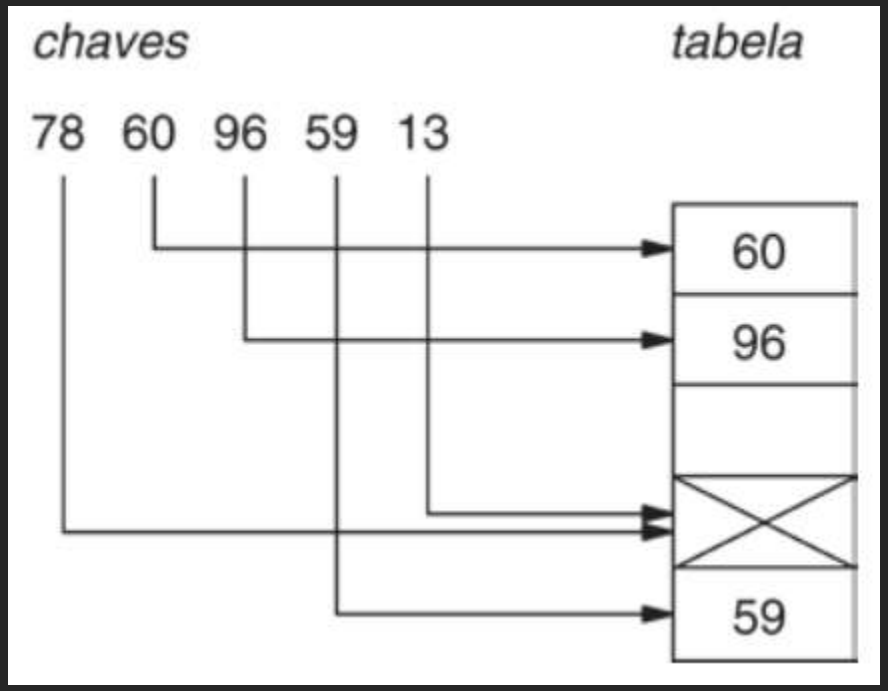
- Em geral, chaves não são valores numéricos de 0 a m-1
  - Podem ser, por exemplo, nomes de pessoas
    - Ainda assim, possuem representação numérica no computador
  - Assume-se, então, que chaves são numéricas
- Por que não utilizar sempre acesso direto com espaços vazios?
  - Ex.: Conjunto com apenas duas chaves com valores de 0 a 999.999 necessita tabela com 1.000.000 compartimentos (só dois ocupados)

Funcionamento

- Solução: transformar cada chave x em um valor no intervalo [0, m-1] usando uma função de dispersão/função de *hashing* h
- Dado o valor de uma chave x, calcula-se h(x), que é o endereço base de x
  - Problema: função h nem sempre é injetora, isto é, pode haver y ≠ x tal que h(y) = h(x)
    - Chaves x e y são sinônimas em relação a h
- Colisão: ao inserir x, compartimento h(x) já está ocupado
  - Tratamento de colisão: procedimento especial para armazenar x em local alternativo

Funcionamento

- Exemplo de colisão com h(x) = x mod 5:
  - Chaves 78 e 13 são sinônimas em relação a h: colisão ao inserir 78 e 13



Funções de dispersão

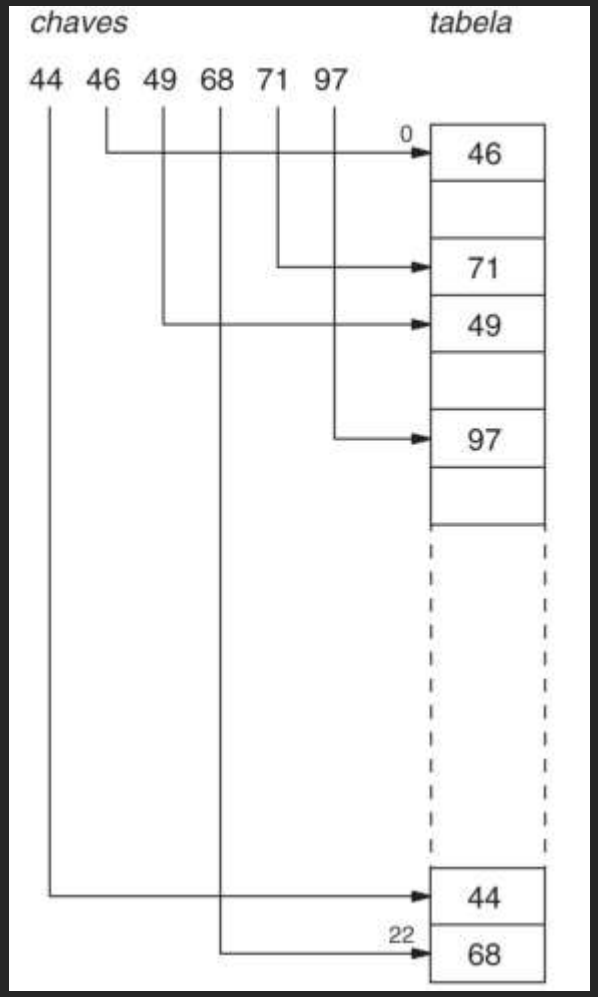
- Função de dispersão h: transforma chave x em endereço-base h(x) da tabela de dispersão
- Uma boa função deve:
  - Produzir baixo número de colisões
    - Nem sempre é possível
  - Ser facilmente computável
  - Ser uniforme: probabilidade de $h(x) = k$ é $1/m$, $k \in [0, m-1]$
    - Difícil de ser testada: distribuição, em geral, é desconhecida
- Funções mais empregadas: método da divisão, da dobra, da multiplicação e da análise dos dígitos

Funções de dispersão

- Método da divisão: bastante empregado por ser fácil e eficiente
- Funcionamento:
  - Chave x é dividida pela dimensão da tabela, m, e o resto da divisão é o endereço base
    - h(x) = x mod m
  - Escolha de m é importante
    - Escolhas ruins de m: par, potência de 2
    - Boa escolha de m: primo não próximo de potência de 2, não possua divisores primos menores que 20

Funções de dispersão

- Exemplo do método da divisão com m = 23

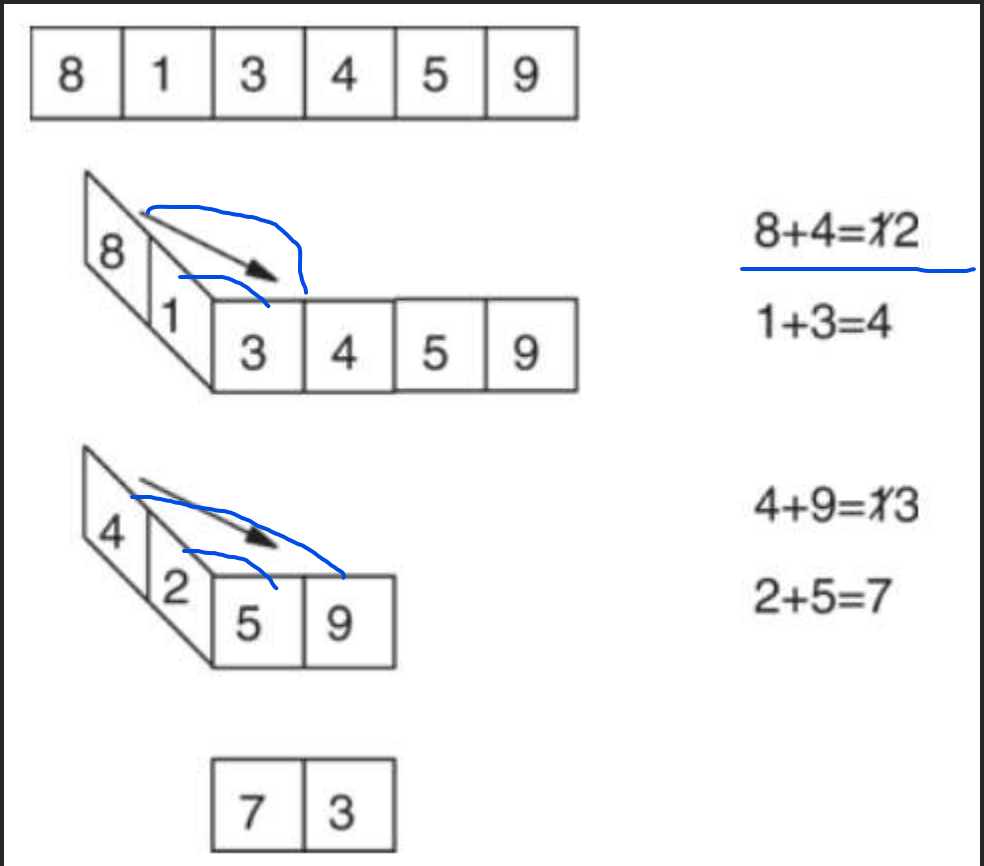


Funções de dispersão

- Método da dobra: chave é reduzida a um endereço-base somando dígitos sobrepostos em dobras
- Funcionamento:
  - Considere chave com os dígitos $d_1,\ldots,d_j$
  - Dobra é realizada no k-ésimo dígito à esquerda
  - Tamanho da chave é reduzido em k dígitos
    - Chave antes da dobra: $d_1,d_2,\ldots,d_k,d_{k+1},\ldots,d_{2k},d_{2k+1},\ldots,d_j$
    - Chave após dobra: $d'_1,\ldots,d'_k,d_{2k+1},\ldots,d_j$
      - $d'_i = d_{k+i} + d_{k-i+1}$ para todo $1 \leq i \leq k$, com soma sem "vai um"
  - Dobras são feitas sucessivamente até obter endereço-base de k dígitos

Funções de dispersão

- Exemplo de dobra com j = 6 e k = 2:



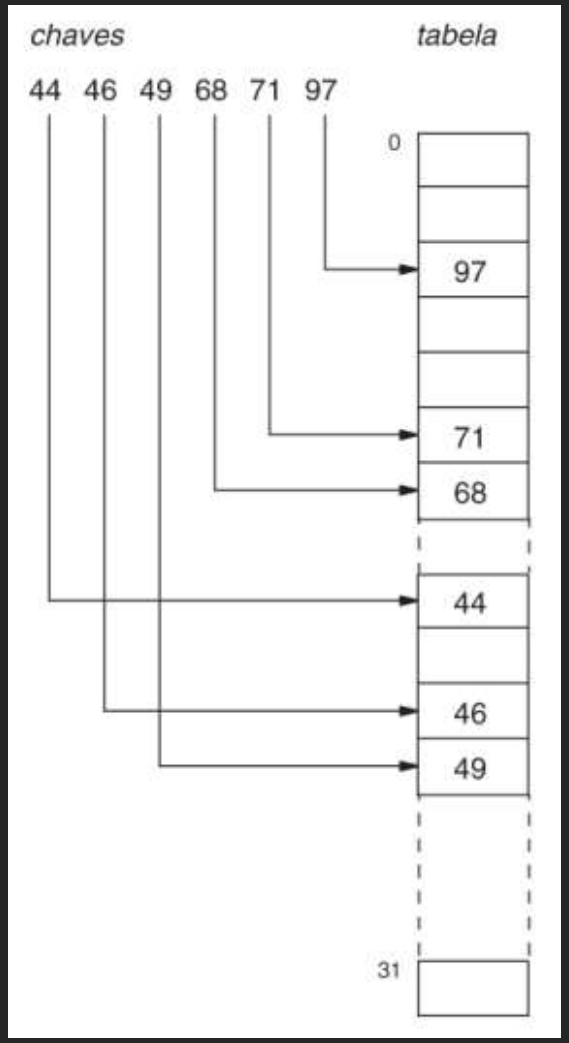
Funções de dispersão

- Alternativa para obter endereço-base de k bits (tabela de dimensão 2^k):
  - Separar chave em diversos campos de k bits
  - Realizar operação lógica bit a bit
    - Operação xor: melhores resultados que and e or

Funções de dispersão

- Exemplo com endereço-base de 5 bits, chaves de 10 bits e operação xor

71 = 00010 00111	endereço-base: (00010) **ouex** (00111) = 00101 = 5
46 = 00001 01110	endereço-base: (00001) **ouex** (01110) = 01111 = 15



Funções de dispersão

- Método da multiplicação: possui variações
  - Variação mais conhecida: método do “meio do quadrado”
- Funcionamento do “meio do quadrado”:
  - Chave é multiplicada por ela mesma (ou por constante)
  - Resultado é armazenado em palavra de b bits
  - Endereço-base de k bits é retirado do meio dos b bits: bits mais significativos e menos significativos são descartados

Funções de dispersão

- Exemplo com chave de 8 bits (b = 16) e k = 10:
  - Chave 207: 11001111
  - 207^2 = 42849: 10100111011000001
  - Endereço-base: 236

Funções de dispersão

- Método da análise dos dígitos: útil quando se conhece previamente as chaves (decimais)
  - Usa função de dispersão para selecionar k melhores dígitos
  - Supondo n chaves com valores uniformemente distribuídos:
    - Há, na média, n/10 chaves com primeiro dígito 0
    - ... n/10 chaves com primeiro dígito 1
    - ...
    - ... n/10 chaves com primeiro dígito 9
    - O mesmo ocorre para o segundo dígito, terceiro dígito, ...

Funções de dispersão

- Suponha que existem n_i chaves com o primeiro dígito i, para $0 \leq i \leq 9$:
  - Somatórios como $\sum_{i=0}^{9}(n_i - n/10)^2$ ou $\sum_{i=0}^{9}|n_i - n/10|$ fornecem o desvio da distribuição do primeiro dígito
    - Quanto menor valor obtido (menor desvio), melhor é o dígito
- Funcionamento:
  - Analise todos os dígitos
  - Endereço-base é composto pelos k melhores dígitos, descartando os demais

Funções de dispersão

- Desvantagens:
  - É necessário conhecer previamente todas as chaves
  - Complexidade O(n) adicional para determinar todos os endereços-base
- Método restrito a aplicações especiais

Funções de dispersão

- Exemplo com k = 1 e as chaves 44, 46, 49, 68, 71 e 97:
  - Como k = 1, há 10 possíveis endereços-base, de 0 a 9
  - Somatório utilizado: $\sum_{i=0}^{9} |n_i - n/10|$
  - Desvio do dígito das dezenas d_1: 7,2
  - Desvio do dígito das unidades d_2: 4,8
  - Utilizando função de dispersão $h(x) = d_2$ não há colisão
    - Com $h(x) = d_1$, há colisões

Tratamento de colisões por encadeamento

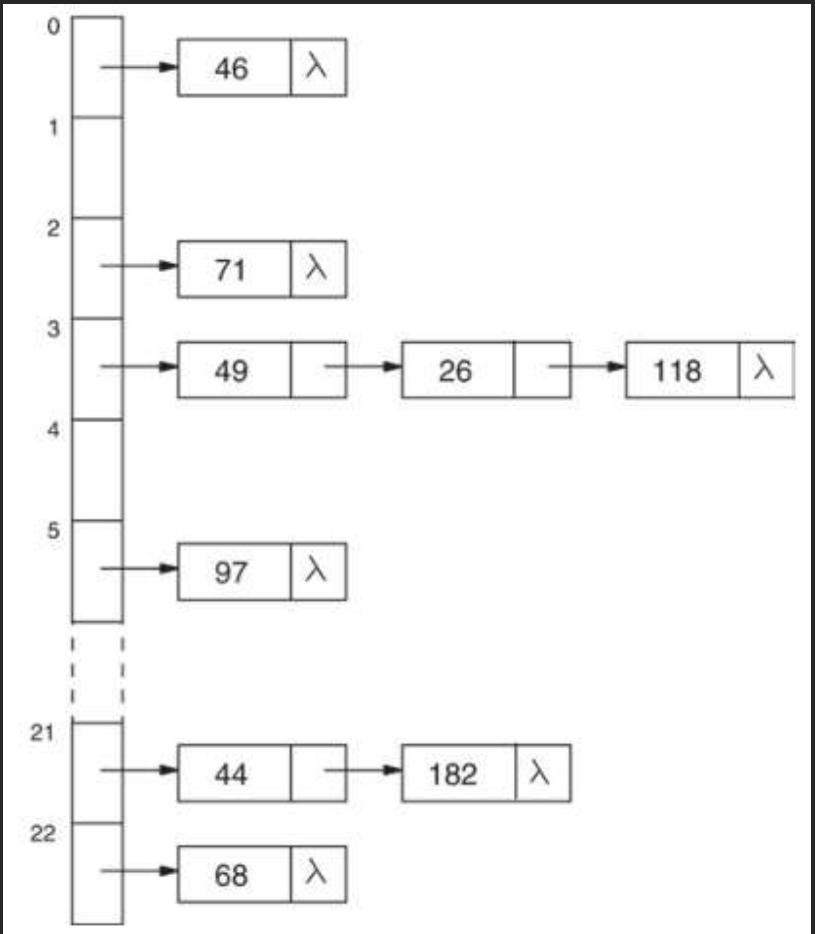
- Fator de carga de uma tabela de dispersão: $\alpha = n/m$
- Método para diminuir colisões: reduzir fator de carga
  - m deve ser adequado para não produzir excessivas colisões
    - Minimiza problema das colisões: não resolve
- Uso de tabelas de dispersão deve prever algum método de tratamento de colisões:
  - Por encadeamento: encadeamento exterior, encadeamento interior
  - Por endereçamento aberto: endereçamento aberto, tentativa lienar, tentativa quadrática, dispersão dupla

Tratamento de colisões por encadeamento

- Encadeamento exterior: solução bastante intuitiva e usada CAI NA PROVA
- Funcionamento:
  - Utiliza m listas encadeadas: uma para cada endereço-base
  - Endereço-base: cabeça da lista encadeada
- Busca:
  - h(x) retorna endereço-base da cabeça da lista
  - Lista é percorrida em busca do elemento x
- Inserção: no início ou final de cada lista

Tratamento de colisões por encadeamento

- Exemplo de encadeamento exterior: h(x) = x mod 23



Tratamento de colisões por encadeamento

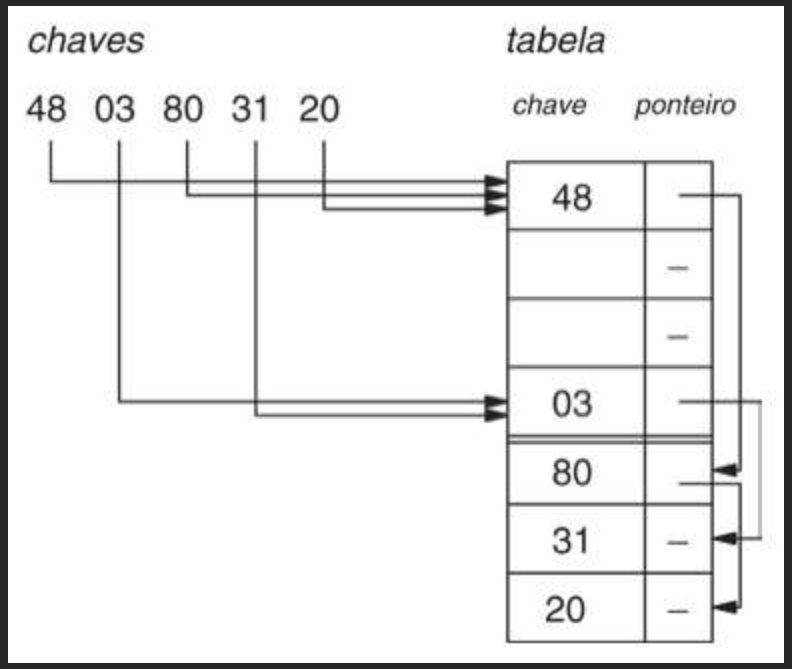
- Complexidade de tempo:
  - Pior caso: comprimento de uma lista pode ser O(n)
    - Busca, inserção e remoção: O(n)
    - Caso médio: $O(\alpha)$, que é constante se n = O(m)

Tratamento de colisões por encadeamento

- Encadeamento interior: lista é armazenada em parte específica da tabela
- Funcionamento:
  - Tabela é dividida em duas partes:
    - Uma parte reservada para os endereços-base: tamanho p
      - Endereços-base: no intervalo de [0, p-1]
    - Outra parte reservada para os sinônimos: tamanho s
  - p e s são fixos, $m = p + s$ e $\alpha \leq 1$
  - Cada entrada da tabela possui dois campos: chave e ponteiro (índice) para próxima chave sinônima

Tratamento de colisões por encadeamento

- Exemplo de encadeamento exterior: h(x) = x mod 4
  - n = 5, p = 4 e s = 3



  - Inclusão de nova chave sinônima: overflow
    - Apesar de haver endereços-base vazios

Tratamento de colisões por encadeamento

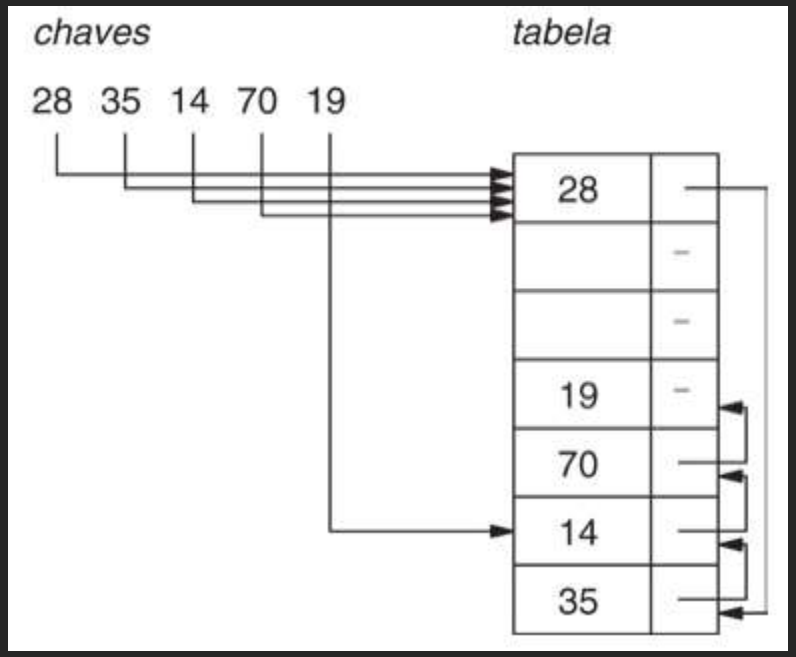
- Tratamento de overflow:
  - Aumentar s e diminuir p: m é mantido
    - Diminui possibilidade de falso overflow (quando há espaço vazio)
    - Eficiência da tabela de dispersão fica menor
- Complexidade de tempo:
  - Pior caso: p = 1 e s = m-1
    - Todas as chaves são sinônimas, independente de h(x)
    - Busca, inserção e remoção: O(n)

Tratamento de colisões por encadeamento

- Outra técnica de encademento interior: não divide a tabela em duas partes
  - Qualquer endereço da tabela pode ser de base ou de colisão (chave sinônima)
  - Produz colisões secundárias: endereço-base de uma chave coincide com endereço outra chave, mas não são sinônimas
- Funcionamento:
  - Se há colisão, chave x é inserida no primeiro espaço vazio d após o compartimento da colisão (ou do fim da tabela para o início)
    - Se houver inserção de chave y onde h(y) = d: fusão das listas com endereços-base h(x) e h(y), gerando menor eficiência

Tratamento de colisões por encadeamento

- Exemplo com colisões secundárias: h(x) = x mod 7
  - Chaves sinônimas: 28, 35, 14 e 70
  - Colisão secundária: entre 14 e 19



Tratamento de colisões por encadeamento

- Tabelas sem remoção: simplifica busca e inserção
- Tabelas com remoção:
  - Remoção:
    - Remoção exige reorganizar tabela: chaves sinônimas/listas fundidas
      - Reorganizar pode gastar muito tempo: fora de cogitação
    - Solução: armazenar o estado do compartimento
      - Vazio: nunca foi utilizado
      - Ocupado: possui chave armazenada
      - Liberado: chave foi removida
        - Compartimento pode ser ocupado novamente no futuro

Tratamento de colisões por encadeamento

- Algoritmos de busca, inserção e remoção:
  - Todas listas são circulares
  - Cada compartimento possui campos chave, estado e pont
    - Estado: ocupado ou não ocupado (vazio ou liberado)
    - Inicialização: T[i].estado = não ocupado e T[i].pont = i, $0 \leq i < m$
  - Busca por x: retorna a e end
    - a = 1: chave foi localizada
      - end: local da chave
    - a = 2: chave não foi localizada
      - end = j, se há compartimento não ocupado j na lista encadeada que contém h(x)
      - end = λ, caso contrário

Tratamento de colisões por encadeamento

- Algoritmo de busca:

```
algoritmo 8.1: Busca por encadeamento interior
procedimento busca(x, end, a)
    a := 0;   end := h(x);   j := λ
    enquanto a = 0 faça
        se T[end].estado = não ocupado então j := end
        se T[end].chave = x e T[end].estado = ocupado então
            a := 1                                  % chave encontrada
        senão end := T[end].pont
            se end = h(x) então
                a := 2;   end := j                  % chave não encontrada
```

Tratamento de colisões por encadeamento

- Algoritmo de inserção:

```
algoritmo 8.2: Inserção por encadeamento interior
busca(x, end, a)
se a ≠ 1 então
        se end ≠ λ então j := end
        senão i := 1;   j := h(x)
                enquanto i ≤ m faça
                        se T[j].estado = ocupado então
                                j := (j + 1) mod m
                                i := i + 1
                        senão i := m + 2   % compartimento não ocupado
                se i = m + 1 então "inserção inválida: overflow"; pare
                temp := T[j].pont                    % fusão de listas
                T[j].pont := T[h(x)].pont
                T[h(x)].pont := temp
        T[j].chave := x                              % inserção de x
        T[j].estado := ocupado
senão "inserção inválida: chave já existente"
```

Tratamento de colisões por encadeamento

- Algoritmo de remoção:

algoritmo 8.3: Remoção por encadeamento anterior
$busca(x, end, a)$
se $a = 1$ **então** $T[end].\,estado := \textit{não ocupado}$
senão "exclusão inválida: chave não existente"

- Complexidade da busca, inserção e remoção: O(n), no pior caso

Tratamento de colisões por endereçamento aberto

- Endereçamento aberto: método que não utiliza ponteiros
  - Economia de espaço
- Funcionamento:
  - Tabela armazena somente chaves
  - Se há colisão, um cálculo determina próximo compartimento a ser analisado
    - Processo é repetido até a chave ser encontrada, ou ser encontrado um compartimento vazio, ou toda tabela ter sido analisada

Tratamento de colisões por endereçamento aberto

- Função de dispersão h(x) deve ser capaz de fornecer até m endereços-base, ao invés de só um: percorrer toda a tabela
  - Função torna-se da forma h(x, k), $0 \leq k \leq m-1$
    - Inicialmente, busca da chave x é feita no compartimento h(x, 0)
    - Se h(x, 0) possui outra chave, busca prossegue em h(x, 1)
    - Se h(x, 1) possui outra chave, busca prossegue em h(x, 2)
    - ...
    - Busca pode prosseguir até h(x, m-1)
  - Sequência de endereços bases h(x, k), $0 \leq k \leq m-1$, é chamada sequência de tentativas

Tratamento de colisões por endereçamento aberto

- Algoritmo de busca por endereçamento aberto:
  - Inicialização: $T[i].chave = \lambda$, $0 \leq i < m$
  - Valores de retorno: a e end
    - a = 1: chave x foi encontrada no compartimento end
    - a = 2: chave x não foi encontrada e compartimento end é vazio
    - a = 3: chave x não foi localizada e tabela está completa

Tratamento de colisões por endereçamento aberto

- Algoritmo:

```
algoritmo 8.4: busca por endereçamento aberto
procedimento busca-aberto(x, end, a)
a := 3;   k := 0
enquanto k < m faça
        end := h(x, k)
        se T[h(x, k)].chave = x então
                a := 1                                  % chave encontrada
                k := m
        senão se T[h(x, k)].chave = λ então
                        a := 2                          % posição livre
                        k := m
                senão k := k + 1
```

Tratamento de colisões por endereçamento aberto

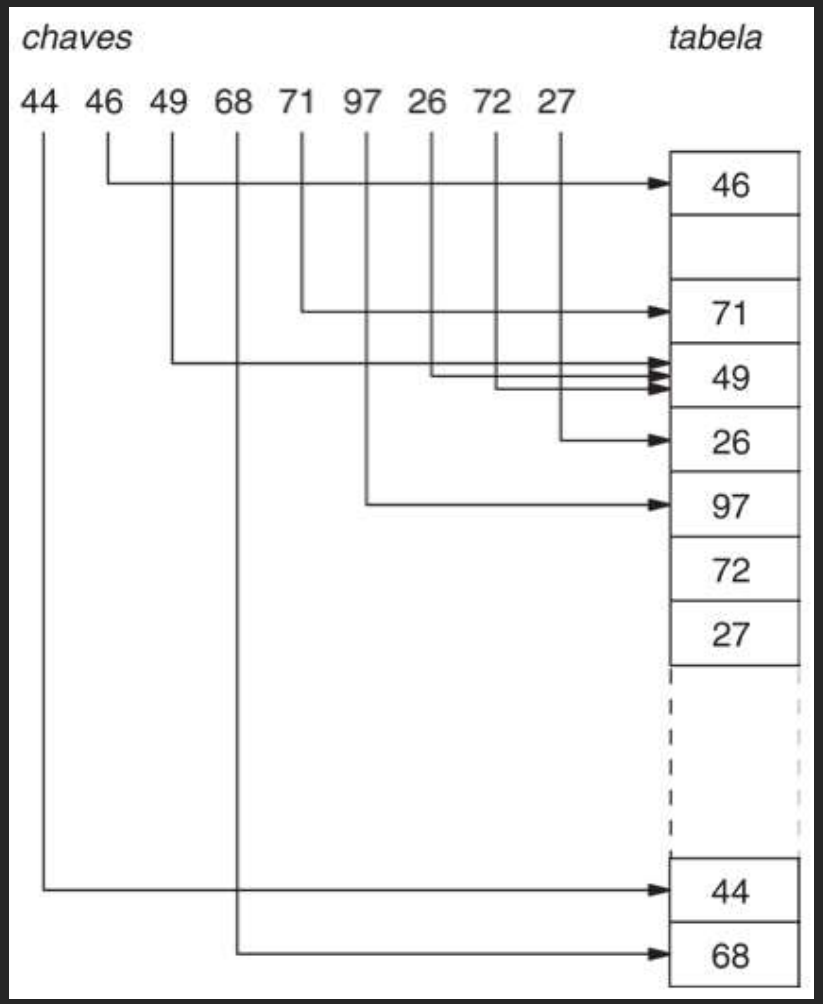
- Algoritmo de inserção: utiliza busca e é bastante simples
- Complexidade:
  - Pior caso da busca e inserção: O(n)
  - Caso médio: O(1) quando fator de carga < 1
- Remoção: problema semelhante ao do encadeamento interior
  - Necessário manter estado do compartimento
  - Também é necessário alterar algoritmos de busca e inserção
- Métodos para determinar sequência de tentativas: tentativa linear, tentativa quadrática, dispersão dupla

Tratamento de colisões por endereçamento aberto

- Tentativa linear: RUIM
  - Suponha que endereço-base da chave x é h'(x)
  - Se há outra chave x' ocupando o mesmo compartimento h'(x), então x é armazenada em h'(x) + 1
  - Se h'(x) + 1 também está ocupado, tenta-se h'(x) + 2, etc.
  - $h(x, k) = (h'(x) + k) \bmod m$, $0 \leq k \leq m-1$

Tratamento de colisões por endereçamento aberto

- Exemplo de tentativa linear:
  - m = 23
  - h'(x) = x mod 23
  - h(x, k) = (x mod 23 + k) mod 23



Tratamento de colisões por endereçamento aberto

- Problema da tentativa linear: produz longos trechos consecutivos ocupados (agrupamento primário)
  - Se existe um agrupamento com j compartimentos consecutivos, a inserção de x que colide com o agrupamento aumenta o tamanho desse agrupamento
  - Quanto maior o agrupamento primário, maior a probabilidade de aumentá-lo
  - Agrupamento longo: tempo de busca alto

Tratamento de colisões por endereçamento aberto

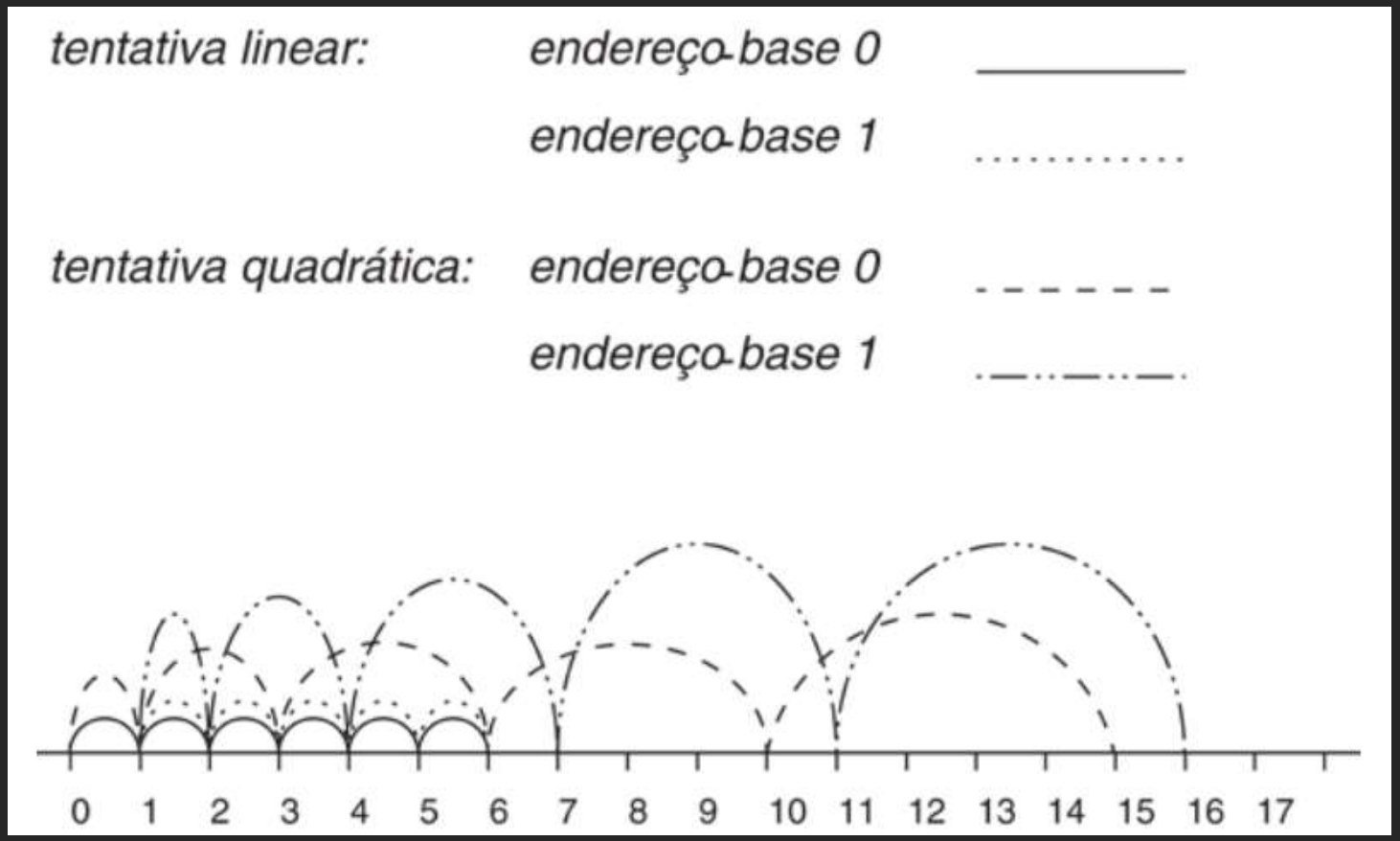
- Tentativa quadrática: mesmo que endereços-base sejam próximos sequência de tentativas diferem
  - $h(x, k) = (h'(x) + c_1k + c_2k^2) \bmod m$, $0 \leq k \leq m-1$, c1 e c2 $\neq 0$ são constantes
    - Escolha de m, c_1 e c_2 deve possibilitar h varrer toda a tabela
      - Equação h recorrente funciona para m potência de 2
        - $h(x, k) = h'(x)$, $k = 0$
        - $h(x, k) = (h(x, k-1) + k) \bmod m$, $0 < k < m$

Tratamento de colisões por endereçamento aberto

- Tentativa quadrática evita agrupamento primário
- Se duas chaves possuem mesma tentativa inicial, as duas sequências são idênticas: agrupamento secundário
  - Degradação menor que o primário

Tratamento de colisões por endereçamento aberto

- Comparação: tentativa linear e quadrática recorrente



Tratamento de colisões por endereçamento aberto

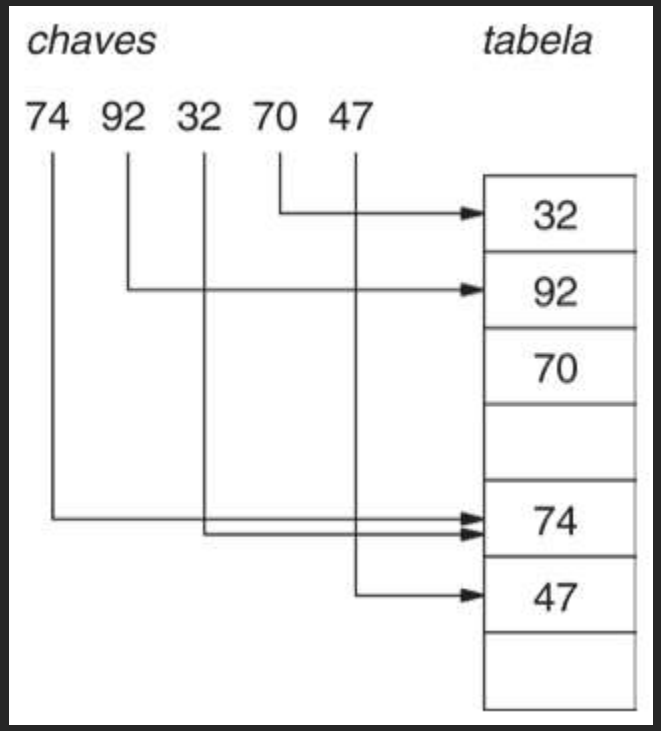
- Dispersão dupla: calcula tentativas usando uma segunda função de dispersão
  - $h(x, k) = (h'(x) + k.h''(x)) \bmod m$, $0 \leq k \leq m-1$
  - Tende a distribuir chaves de melhor forma que tentativas linear ou quadrática
    - Se $x \neq y$, mas $h'(x) = h'(y)$, então sequência de tentativas geradas pela tenativa linear (ou quadrática) são idênticas
    - Na dispersão dupla, as sequências são idênticas somente se $h'(x) = h'(y)$ e $h''(x) = h''(y)$

Tratamento de colisões por endereçamento aberto

- h''(x) e m devem ser primos entre si para que toda a tabela seja varrida
  - Escolhas comuns de h'' e m:
    - Escolha de m potência de 2 e h'' que produza números ímpares
    - Escolha de m primo

Tratamento de colisões por endereçamento aberto

- Exemplo de dispersão dupla com m = 7:
  - h(x, k) = (x mod 7 + k(1 + x mod 5)) mod 7



Exercícios

- Livro Szwarcfiter:
  - 8.1, 8.2, 8.4, 8.5, 8.7, 8.9, 8.10 e 8.11